ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

(EMSI) - Les Orangers

Filière Génie Informatique

Rapport De Projet

Implémentation et comparaison d'architectures de Deep Learning : De la classification tabulaire à la modélisation séquentielle

MLP, CNN, RNN, LSTM, GRU et Architectures Hybrides

RÉALISÉ PAR :

INAS MOURID

ENCADRÉ PAR :

Mme. HIDILA Zineb

Année Universitaire 2025/2026

Remis le : 21 Juin 2026

Document confidentiel - Propriété intellectuelle de l'EMSI

Remerciements

À l’issue de ce projet, il m’est agréable d’exprimer ma profonde gratitude à ceux qui ont contribué à son aboutissement.

Je tiens à exprimer mes plus sincères remerciements à mon encadrante, **Madame HIDILA Zineb**, pour la confiance qu’elle m’a témoignée, pour ses précieux conseils, sa disponibilité constante et la rigueur scientifique qu’elle m’a transmise tout au long de ce projet. Son aide a été déterminante dans la réussite de ce travail.

Je souhaite également remercier le corps professoral de l'EMSI pour la qualité de l’enseignement dispensé, qui a constitué le socle théorique nécessaire à la réalisation de ce projet.

Enfin, j’adresse mes remerciements à ma famille et à mes proches, dont le soutien moral a été une source de motivation quotidienne durant toute la durée de mon cursus.

Résumé

Ce rapport présente l'implémentation et l'évaluation de plusieurs architectures de Deep Learning, couvrant les perceptrons multicouches (MLP) pour les données tabulaires, les réseaux de neurones convolutifs (CNN) pour l'imagerie, et les réseaux récurrents (LSTM/GRU) pour les séquences. L'objectif principal était de maîtriser le cycle complet de développement : du prétraitement des données à l'optimisation des hyperparamètres, en passant par l'entraînement et l'évaluation. Nous mettons en lumière l'importance cruciale des techniques de régularisation, de la gestion du déséquilibre des classes et du choix des métriques pour garantir une généralisation robuste. Les résultats expérimentaux confirment qu'une adéquation parfaite entre l'architecture et la nature des données est indispensable. Ce travail se conclut par une réflexion sur les bonnes pratiques en ingénierie logicielle appliquée au *Machine Learning*, en insistant sur la modularité, la reproductibilité et l'esprit critique dans l'interprétation des résultats.

Abstract

Abstract: This report explores the implementation and evaluation of various Deep Learning architectures, ranging from Multilayer Perceptrons (MLP) for tabular data to Convolutional Neural Networks (CNN) for image recognition, and Recurrent Neural Networks (LSTM/GRU) for sequence modeling. The objective of this study was to master the end-to-end development cycle—from data preprocessing and feature engineering to model training and hyperparameter optimization. We emphasize the critical importance of regularisation techniques, class balancing, and performance metrics in ensuring robust generalization. Our experimental results demonstrate that adapting the architecture to the specific nature of the data is paramount. The study concludes with a reflection on the best practices in software engineering applied to machine learning, emphasizing modularity, reproducibility, and critical analysis of model outputs.

Table des matières

Remerciements 2

Résumé 3

Abstract 4

Introduction 7

1.1 **Vue d'ensemble des architectures 7**

2. Préparation des données 8

2.1 **Exploration et compréhension du dataset 8**

2.2 **Prétraitement 8**

2.2.1 Données tabulaires (MLP) 8

2.2.2 Images (CNN) 8

2.2.3 Séquences/Signaux (RNN/LSTM/GRU) 9

2.3 **Séparation des données 9**

3.Architectures en détail 9

3.1 Perceptron Multicouche (MLP) 10

3.1.1 Structure type 10

3.1.2 Points d'attention 10

3.2 Réseaux de Neurones Convolutifs (CNN) 11

3.2.1 Structure type (CNN personnalisé) 11

3.2.2 Transfert d'apprentissage 11

3.2.3 Augmentation de données 11

3.3 Réseaux Récurrents (RNN, LSTM, GRU) 11

3.3.2 Comparaison des variantes 12

3.3.3 Configuration type (LSTM) 12

3.4 Architectures hybrides 12

3.4.1 CNN + LSTM **13**

3.4.2 CNN + MLP (Fusion multimodale) 13

3.4.3 Stratégies de fusion 13

4 Processus d'entraînement 14

4.1 Fonctions de perte 14

4.2 Gestion du déséquilibre des classes 14

4.3 Optimiseurs 14

4.4 Schedulers de learning rate **14**
4.5 Régularisation **14**
5 Guide des hyperparammètres 15
5.1 Hyperparammètres clés par architecture **15**
5.1.2 MLP **15**
5.1.3 CNN **15**
5.1.4 RNN / LSTM / GRU **15**
5.2 Stratégies de recherche d'hyperparammètres **13**
6 Métriques d'évaluation 15
6.1 Métriques de classification **15**
6.2 Visualisations obligatoires **15**
6.3 Diagnostic par les courbes d'apprentissage **16**
7 Interprétabilité des modèles 16
8 Bonnes pratiques et erreurs fréquentes 17
8.1 Checklist avant entraînement **17**
8.2 Erreurs fréquentes **17**
8.3 Structure type du code d'entraînement **17**
9 Guide de rédaction du rapport 18
9.1 Structure attendue **18**
9.2 Critères de qualité rédactionnelle **20**
9.3 Ce qu'il faut éviter **20**
10 Références bibliographiques recommandées 20
10.1 Articles fondateurs **20**
10.2 Ressources pédagogiques **20**

1. Introduction

Le domaine du *Deep Learning* a radicalement transformé l'ingénierie des systèmes intelligents en permettant l'extraction automatique de hiérarchies de caractéristiques complexes à partir de données brutes. Ce projet de fin de module s'inscrit dans cette dynamique en explorant trois paradigmes fondamentaux, chacun adapté à une structure géométrique de donnée spécifique. L'objectif central de ce travail est de démontrer une maîtrise approfondie de l'ingénierie PyTorch, en articulant la préparation rigoureuse des jeux de données, l'implémentation d'architectures spécialisées et l'analyse critique des performances. Ce rapport détaille non seulement la mise en œuvre technique de ces modèles — allant du Perceptron Multicouche aux réseaux récurrents — mais propose également une réflexion scientifique sur l'adéquation entre la nature statistique des données et le choix de l'architecture, répondant ainsi aux exigences de rigueur académique du cursus.

1.2 Vue d'ensemble des architectures

La conception de systèmes de Deep Learning performants repose sur une adéquation précise entre la topologie du réseau et la structure de l'information traitée. Dans le cadre de ce projet, nous avons articulé notre approche autour de trois piliers architecturaux complémentaires :

1. **Le Perceptron Multicouche (MLP) :** Utilisé pour le traitement des données tabulaires (dataset *Breast Cancer Wisconsin*), cette architecture exploite la capacité des couches denses à approximer des fonctions non-linéaires complexes. Elle constitue notre base de référence pour l'apprentissage supervisé sur des attributs extraits manuellement.
2. **Les Réseaux de Neurones Convolutifs (CNN) :** Déployés pour la vision par ordinateur (dataset *Fashion-MNIST*), ces réseaux tirent profit de l'invariance par translation et de la localité spatiale des pixels. L'utilisation d'opérations de convolution et de pooling permet d'extraire des hiérarchies de primitives visuelles (contours, textures, formes) avec une efficacité paramétrique supérieure au MLP.
3. **Les Réseaux Récurrents (RNN, LSTM, GRU) et Seq2Seq :** Appliqués aux données séquentielles (dataset *Tatoeba*), ces modèles sont conçus pour modéliser les dépendances temporelles de longue durée. Par le biais d'un mécanisme d'encodeur-décodeur, ils permettent de résoudre des problématiques de transduction de séquences, où la sortie dépend de l'intégralité du contexte passé.

Chaque architecture a été mise en œuvre en respectant les bonnes pratiques de PyTorch : gestion explicite des tenseurs, suivi des gradients via *Autograd* et optimisation ciblée des hyperparamètres, afin de garantir une robustesse expérimentale conforme aux standards de l'ingénierie moderne.

2. Préparation des données

2.1 Exploration et compréhension du dataset

La phase d'exploration est une étape déterminante pour identifier les spécificités statistiques de chaque domaine d'application. Cette étape permet d'ajuster les stratégies de prétraitement afin de maximiser la capacité d'apprentissage des modèles.

- **Données Tabulaires (Dataset *Breast Cancer Wisconsin*) :** Ce jeu de données se compose de 30 caractéristiques numériques calculées à partir d'images numérisées de prélèvements cellulaires. L'exploration révèle des attributs présentant des échelles de grandeur variées (ex: rayon, texture, périmètre). La classification binaire (tumeur maligne ou bénigne) présente une distribution légèrement déséquilibrée, nécessitant une attention particulière lors du découpage des données.
- **Données Images (Dataset *Fashion-MNIST*) :** Ce corpus contient 70 000 images en niveaux de gris (résolution 28x28 pixels), réparties en 10 classes de produits de mode (t-shirts, chaussures, sacs, etc.). L'analyse exploratoire montre une forte corrélation spatiale entre les pixels adjacents, justifiant l'utilisation d'architectures convolutives capables de capturer cette structure locale.
- **Données Séquentielles (Corpus *Tatoeba* - Français/Anglais) :** Ce dataset textuel est constitué de paires de phrases traduisibles. La compréhension de ce jeu de données met en évidence des contraintes de longueur variable et une richesse lexicale propre à chaque langue. L'exploration porte ici sur la distribution de la longueur des séquences, afin de définir les paramètres de troncation et de padding nécessaires à la mise en forme des tenseurs d'entrée.

2.2 Prétraitement

Le prétraitement des données est une étape cruciale pour assurer la stabilité numérique des gradients et la convergence des modèles. Chaque architecture impose des contraintes spécifiques sur la forme et la distribution des tenseurs d'entrée.

2.2.1 Données tabulaires (MLP)

Pour le dataset *Breast Cancer*, le prétraitement se concentre sur l'harmonisation des échelles de features.

- **Standardisation :** Nous avons utilisé le StandardScaler de Scikit-Learn pour centrer les données (moyenne = 0) et les réduire (variance = 1). Cette étape est indispensable pour éviter que des variables à forte échelle ne dominent la fonction de coût, ce qui ralentirait la descente de gradient.
- **Encodage :** Les labels de sortie ont été convertis en format numérique binaire (0 ou 1) pour être compatibles avec la fonction de perte BCELoss (Binary Cross-Entropy

2.2.2 Images (CNN)

Le prétraitement pour *Fashion-MNIST* vise à transformer les données brutes en tenseurs normalisés, prêts pour les couches de convolution.

- **Normalisation :** Les valeurs des pixels, initialement comprises entre [0, 255], ont été normalisées dans l'intervalle [0, 1] (voire [-1, 1] selon la configuration) pour garantir que les activations restent dans une plage optimale pour les fonctions d'activation (ReLU).
- **Tenseurification :** Les images ont été converties en tenseurs PyTorch avec une dimension de canal ajoutée explicitement ([Batch, Channel, Height, Width]), nécessaire pour que les couches nn.Conv2d puissent effectuer les opérat ions de balayage spatial.

2.2.3 Séquences/Signaux (RNN/LSTM/GRU)

Le traitement du langage naturel sur le corpus *Tatoeba* nécessite une transformation discrète vers un espace vectoriel dense.

- **Tokenisation et Vocabulaire :** Nous avons procédé à une tokenisation par mots. Un vocabulaire a été construit pour mapper chaque mot unique à un index entier unique.
- **Gestion des séquences :** * **Tokens spéciaux :** Ajout systématique des marqueurs <SOS> (Start of Sentence) et <EOS> (End of Sentence) pour définir les limites de la séquence.
    - **Padding :** Étant donné que les phrases ont des longueurs variables, nous avons appliqué un padding pour aligner toutes les séquences à une longueur fixe, afin de permettre le traitement par lots (*batching*). Le token <PAD> est ignoré lors du calcul de la perte.
    - **Embedding :** Les indices sont ensuite projetés dans un espace vectoriel de dimension *d* via une couche d'Embedding, permettant au modèle d'apprendre des relations sémantiques entre les mots.

2.3 Séparation des données

La séparation des jeux de données est une étape fondamentale pour garantir la capacité de généralisation des modèles et prévenir le surapprentissage (*overfitting*). Conformément aux bonnes pratiques en apprentissage supervisé, nous avons adopté une stratégie de découpage systématique pour chacun de nos datasets.

- **La stratégie de split :** Nous avons divisé chaque jeu de données en trois ensembles distincts :
    - **Ensemble d'entraînement (*Training Set*) :** Représentant généralement 70% à 80% des données, il est utilisé exclusivement pour l'ajustement des poids du modèle via la rétropropagation.
    - **Ensemble de validation (*Validation Set*) :** Représentant 10% à 15% des données, il sert de baromètre durant l'entraînement pour ajuster les hyperparamètres et surveiller le phénomène de surapprentissage sans pour autant influencer directement l'optimisation des poids.
    - **Ensemble de test (*Test Set*) :** Représentant les 10% à 15% restants, il est conservé isolément jusqu'à la phase finale. Il constitue le "standard de vérité" permettant une évaluation impartiale des performances du modèle sur des données totalement inédites.
- **Précautions méthodologiques :**
    - **Stratification :** Pour le dataset *Breast Cancer*, nous avons utilisé une séparation stratifiée (*Stratified Split*). Cette technique garantit que la proportion des classes (maligne/bénigne) est identique dans les trois sous-ensembles, évitant ainsi un déséquilibre qui biaiserait l'évaluation.
    - **Indépendance :** Pour le corpus *Tatoeba*, une attention particulière a été portée à l'absence de chevauchement entre les paires de phrases de l'entraînement et du test, garantissant que le modèle apprend réellement à traduire et non à mémoriser des phrases par cœur.

3. Architectures en détail

Cette section détaille la conception architecturale de nos modèles, en mettant l'accent sur la cohérence entre la topologie du réseau et les propriétés statistiques des données traitées.

3.1 Perceptron Multicouche (MLP)

Le Perceptron Multicouche est l'architecture fondamentale de notre étude sur les données tabulaires. Il repose sur l'empilement successif de couches linéaires suivies de fonctions d'activation non-linéaires, permettant de modéliser des surfaces de décision complexes.

3.1.1 Structure type

Notre implémentation, conçue pour le dataset *Breast Cancer*, suit une architecture de type "Feed-Forward" :

- **Couche d'entrée :** Dimensionnée selon le nombre de caractéristiques du dataset (30 neurones d'entrée).
- **Couches cachées** : Nous avons articulé le réseau autour de couches denses (Hidden Layers) équipées d'une fonction d'activation ReLU (*Rectified Linear Unit*). Le choix de ReLU est stratégique pour limiter l'évanouissement du gradient (*vanishing gradient*) tout en introduisant la non-linéarité nécessaire.
- **Couche de sortie :** Dotée d'une activation Sigmoïde pour les tâches de classification binaire, fournissant une probabilité d'appartenance à la classe positive (tumeur maligne).

3.1.2 Points d’attention

L'implémentation d'un MLP sur des données réelles nécessite une vigilance particulière sur plusieurs points critiques :

- **Initialisation des poids** : Comme observé lors de nos expérimentations, une initialisation constante bloque la dynamique d'apprentissage par symétrie. Nous avons retenu l'initialisation de Xavier (Glorot), qui ajuste la variance des poids en fonction du nombre de connexions entrantes et sortantes, garantissant ainsi une distribution stable des activations à travers les couches.
- **Gestion des paramètres** : L'inspection via named_parameters() est cruciale pour valider l'adéquation des dimensions des matrices de poids ($W \in \mathbb{R}^{in \times out}$) avec le flux des données. Chaque paramètre doit être explicitement enregistré dans le graphe de calcul pour permettre la rétropropagation.
- **Sur-paramétrisation et Overfitting :** Étant donné la haute dimensionnalité, nous avons veillé à ne pas concevoir un réseau inutilement profond pour éviter le surapprentissage. Le risque majeur ici est la mémorisation du bruit statistique plutôt que l'apprentissage des motifs sous-jacents, particulièrement lorsque le nombre d'échantillons est limité.

Stabilité numérique : Le choix de l'optimiseur (Adam) a été déterminant pour ajuster dynamiquement le taux d'apprentissage, évitant ainsi les oscillations autour des minima locaux de la fonction de coût.

3.2 Réseaux de Neurones Convolutifs (CNN)

Les CNN constituent la réponse architecturale optimale aux défis de la vision par ordinateur. Contrairement au MLP, le CNN exploite la structure spatiale bidimensionnelle des images grâce à des opérateurs de convolution qui préservent la localité des pixels.

3.2.1 Structure type (CNN personnalisé)

Notre modèle LeNetAmeliore, appliqué au dataset *Fashion-MNIST*, repose sur une hiérarchie d'extraction de caractéristiques :

- **Blocs de Convolution :** Composés de couches nn.Conv2d suivies d'une activation ReLU. Ces couches permettent de détecter des motifs locaux (bords, textures, formes) grâce à des filtres apprenables.
- **Opérations de Pooling** : L'utilisation de nn.MaxPool2d réduit la dimensionnalité spatiale des cartes de caractéristiques (*feature maps*), offrant une invariance par translation et limitant le coût computationnel.
- **Tête de classification** : Après aplatissement (*flattening*), les caractéristiques extraites sont transmises à des couches denses pour aboutir à une classification finale sur 10 classes**.**

3.2.2 Transfert d'apprentissage

Bien que notre implémentation sur *Fashion-MNIST* soit construite *from scratch* pour des besoins pédagogiques, nous avons étudié le concept de Transfer Learning. Cette technique consiste à utiliser un réseau pré-entraîné sur un dataset massif (tel qu'ImageNet) pour extraire des caractéristiques génériques.

- Avantage : Cela permet de bénéficier de la puissance d'abstraction de modèles profonds (comme ResNet ou VGG) tout en n'entraînant que la couche de classification finale, ce qui est crucial lorsque le volume de données local est limité.

3.2.3 Augmentation de données

Pour renforcer la robustesse de notre CNN face aux variations des images, nous avons implémenté des techniques d'augmentation de données (*Data Augmentation*) :

- **Transformations géométriques** : Rotations légères, retournements horizontaux (*horizontal flips*) et décalages (*random crops*).
- **Impact** : Ces transformations injectent du bruit positif dans le processus d'entraînement, forçant le modèle à généraliser davantage ses caractéristiques et réduisant drastiquement le risque de surapprentissage (*overfitting*). Le modèle n'apprend plus la position précise d'un pixel, mais la forme globale de l'objet, ce qui est vital pour la classification de vêtements.

3.3 Réseaux Récurrents (RNN, LSTM, GRU)

Les réseaux récurrents sont conçus pour traiter des données où l'ordre des éléments est porteur de sens. Contrairement aux réseaux classiques, ils introduisent une dépendance temporelle par le biais d'un état caché qui agit comme une "mémoire" du contexte passé.

3.3.1 Comparaison des variantes

L'évolution des architectures récurrentes répond directement au problème de la disparition du gradient (*vanishing gradient*) lors du processus de *Backpropagation Through Time* (BPTT).

- **RNN classique** : Simple mais limité par sa difficulté à retenir des dépendances sur de longues séquences.
- **LSTM (Long Short-Term Memory)** : Introduit une structure de "cellule" avec trois portes (entrée, oubli, sortie). Ces portes permettent au réseau de réguler sélectivement l'information à conserver ou à effacer, offrant une bien meilleure stabilité sur le long terme.
- **GRU (Gated Recurrent Unit)** : Une version simplifiée du LSTM qui fusionne les portes d'entrée et d'oubli en une seule "porte de mise à jour" (*update gate*). Cette architecture offre des performances comparables au LSTM avec un coût computationnel réduit et une convergence souvent plus rapide.

3.3.2 Configuration type (LSTM/GRU)

Dans le cadre de notre projet Seq2Seq, nous avons configuré notre modèle (basé sur GRU) pour optimiser le transfert d'information :

- **Encodeur** : Une couche récurrente qui traite la séquence source et produit un vecteur de contexte représentant la "pensée" de la phrase.
- **Décodeur :** Une seconde couche récurrente qui utilise ce vecteur de contexte pour générer, pas à pas, la séquence cible.
- **Paramétrage :**
    - **Dimension cachée (*hidden_dim*)** : Fixée pour équilibrer la capacité de mémorisation du modèle et le risque de surapprentissage.
    - **Dropout :** Appliqué entre les couches récurrentes pour régulariser le modèle et éviter qu'il ne s'appuie trop fortement sur des neurones spécifiques, ce qui améliore la robustesse lors de la traduction de phrases inédites.
    - **Gradient Clipping :** Crucial dans cette architecture, nous avons implémenté une borne sur la norme des gradients (clip_grad_norm_) pour empêcher les explosions de gradients, fréquentes dans les architectures récurrentes profondes.

3.4 Architectures hybrides

Lorsque les données présentent une complexité multimodale ou une structure spatio-temporelle imbriquée, une architecture unique peut s'avérer insuffisante. Les architectures hybrides permettent de fusionner les capacités d'extraction spécialisées pour une représentation globale plus riche.

3.4.1 CNN + LSTM

Cette architecture est particulièrement efficace pour des tâches de légendage d'image (*image captioning*) ou d'analyse vidéo.

- **Fonctionnement** : Le CNN agit comme un encodeur spatial, transformant l'image en une carte de caractéristiques haute dimensionnelle. Cette sortie est ensuite aplatie et traitée comme une séquence d'entrées par le LSTM.
- **Intérêt** : Le CNN capture le "quoi" (contenu visuel) et le LSTM capture le

"comment" (dépendances séquentielles ou temporelles), permettant de décrire une scène visuelle par une suite de mots cohérents.

3.4.2 CNN + MLP (Fusion multimodale)

Utilisée pour les systèmes traitant simultanément des données tabulaires et visuelles (par exemple, le diagnostic médical assisté par ordinateur intégrant l'image radiologique et les données cliniques du patient).

- **Fonctionnement :** Les deux branches (CNN pour l'image, MLP pour les données tabulaires) sont entraînées en parallèle. Les vecteurs de caractéristiques issus de chaque branche sont concaténés avant de passer à travers une couche finale de classification commune.
- **Intérêt** : Cette fusion multimodale permet au modèle de prendre des décisions basées sur une vision globale du patient, là où un modèle unique serait aveugle à une partie de l'information.

3.4.3 Stratégies de fusion

Le succès d'une architecture hybride repose sur la méthode de fusion des vecteurs de caractéristiques :

- **Concaténation (Early/Late Fusion) :** La méthode la plus courante, consistant à juxtaposer les vecteurs V_{cnn} et V_{mlp} pour former un vecteur concaténé V_{total}.
- **Fusion par addition ou produit élémentaire** : Une méthode plus légère qui préserve la dimensionnalité mais nécessite une normalisation rigoureuse des activations entre les deux branches pour éviter qu'une modalité ne domine l'autre (gradient vanishing/explosion).
- **Attention croisée (*Cross-Attention*)** : Une approche avancée où une modalité pondère l'importance de l'autre, permettant au modèle de se concentrer dynamiquement sur les caractéristiques visuelles les plus pertinentes par rapport aux données tabulaires fournies.

4. Processus d'entraînement

L'entraînement est la phase où l'architecture apprend les représentations optimales. Ce processus exige une configuration rigoureuse pour assurer la convergence de la fonction de coût vers un minimum global.

4.1 Fonctions de perte (*Loss Functions*)

Le choix de la fonction de perte est dicté par la nature de la tâche de sortie :

- **Classification binaire (MLP - Breast Cancer)** : Nous avons utilisé la BCELoss (Binary Cross-Entropy), idéale pour modéliser une probabilité d'appartenance à deux classes.
- **Classification multi-classe (CNN - Fashion-MNIST)** : La CrossEntropyLoss a

été retenue. Elle combine un LogSoftmax et une NLLLoss pour maximiser la log-vraisemblance de la classe correcte.

- **Traduction (Seq2Seq - Tatoeba)** : Une CrossEntropyLoss avec gestion du masque de padding est utilisée pour comparer les distributions de probabilités des tokens générés à chaque pas de temps.

4.2 Gestion du déséquilibre des classes

Lorsque la distribution des classes est inégale, le modèle a tendance à favoriser la classe majoritaire.

- **Stratégies appliquées :** Nous avons recours au poids des classes (*class weights*) dans la fonction de perte. En augmentant la pénalité associée aux erreurs commises sur la classe minoritaire, nous forçons le réseau à porter une attention accrue aux cas rares, améliorant ainsi le score de rappel (*Recall*).

4.3 Optimiseurs

Nous avons privilégié l'optimiseur Adam (*Adaptive Moment Estimation*). Contrairement à une descente de gradient stochastique (SGD) classique, Adam adapte le taux d'apprentissage de chaque paramètre individuellement en utilisant les moments du premier et du second ordre du gradient. Cela permet une convergence rapide et une grande robustesse face aux surfaces de perte irrégulières.

4.4 Schedulers de learning rate

Pour affiner la convergence, nous avons implémenté des *schedulers* qui modifient le taux d'apprentissage en cours d'entraînement :

- Réduction sur plateau (*ReduceLROnPlateau*) : Le taux d'apprentissage est réduit automatiquement lorsque la perte de validation stagne, permettant ainsi au modèle d'atteindre des minima plus précis en fin d'entraînement.

4.5 Régularisation

La régularisation est notre principal levier contre le surapprentissage (*overfitting*) :

- Dropout : Insertion de couches de Dropout (typiquement à 0.5) dans les MLP et les CNN, forçant le réseau à apprendre des représentations redondantes et robustes.
- Gradient Clipping : Spécifiquement pour les modèles RNN/Seq2Seq, nous avons imposé une borne sur la norme des gradients. Cela empêche l'explosion des gradients (*exploding gradients*), assurant la stabilité de la mémoire à long terme.

5. Guide des hyperparamètres

La performance d'un modèle de *Deep Learning* est intimement liée au choix judicieux de ses hyperparamètres. Ces derniers ne sont pas appris par le modèle mais fixés a priori ; ils dictent la dynamique de l'apprentissage et la capacité de généralisation.

5.1 Hyperparamètres clés par architecture

5.1.1 MLP

Pour le perceptron multicouche, nous avons concentré nos réglages sur :

- Nombre de couches et neurones par couche : Un équilibre entre la complexité de la fonction de décision et le risque de surapprentissage.
- Taux d'apprentissage (*Learning Rate*) : Un paramètre critique ; une valeur trop élevée provoque une divergence, tandis qu'une valeur trop faible conduit à une stagnation prématurée.
- Fonction d'activation : Le choix de ReLU s'est imposé pour sa capacité à maintenir un gradient actif sur de larges domaines**.**

5.1.2 CNN

Les hyperparamètres spécifiques aux réseaux convolutifs incluent :

- Taille du noyau (*Kernel size*) : Généralement fixée à 3×3 ou 5×5 pour extraire des caractéristiques locales tout en limitant le nombre de paramètres.
- Pas (*Stride*) et Padding : Utilisés pour contrôler la réduction de dimension spatiale lors du passage dans les couches de convolution et de pooling.
- Profondeur du réseau : Le nombre de blocs convolutionnels détermine le niveau d'abstraction des caractéristiques (des bords aux formes complexes**).**

5.1.3 RNN / LSTM / GRU

Pour les modèles séquentiels, les paramètres deviennent plus complexes à stabiliser :

- Dimension de l'espace latent (*Hidden Size*) : Définit la capacité de "mémoire" du réseau. Une dimension trop élevée augmente inutilement la complexité computationnelle et le risque de surapprentissage.
- Dimension de l'Embedding : Définit la richesse sémantique des vecteurs de mots.
- Nombre de couches récurrentes : Plus le réseau est profond, plus il est difficile à entraîner, nécessitant impérativement une stratégie de régularisation comme le Gradient Clipping mentionné précédemment.

5.2 Stratégies de recherche d’hyperparamètres

La recherche d'une configuration optimale a suivi une méthode itérative :

1. **Exploration manuelle par tâtonnement :** Cette approche nous a permis de comprendre l'influence directe de chaque paramètre sur la courbe de perte.
2. **Monitorage de la validation :** Chaque changement d'hyperparamètre était validé par une analyse des courbes d'apprentissage pour identifier rapidement si le modèle tendait vers l'overfitting ou sous-performait**.**

6. Métriques d’évaluation

L'évaluation quantitative est le seul moyen de valider objectivement la performance d'un modèle. Elle permet de mesurer non seulement la précision globale, mais aussi la capacité de généralisation

sur des données non vues.

6.1 Métriques de classification

Pour quantifier les résultats, nous avons utilisé un ensemble de métriques complémentaires :

- Accuracy : Mesure la proportion de prédictions correctes. Bien qu'intuitive, elle est parfois trompeuse en cas de déséquilibre de classes.
- Precision et Recall : Cruciales pour les datasets comme *Breast Cancer* (où le coût d'un faux négatif est critique). La Precision évalue la fiabilité des prédictions positives, tandis que le Recall mesure la capacité du modèle à identifier tous les cas positifs.
- F1-Score : La moyenne harmonique entre la précision et le rappel. C'est la métrique de choix pour obtenir une vision équilibrée de la performance du modèle.
- Score BLEU : Spécifiquement pour notre modèle Seq2Seq, nous avons utilisé le score BLEU unigramme. Il compare la similarité entre la traduction générée et la traduction de référence, en intégrant une pénalité de brièveté pour éviter les sorties trop courtes.

6.2 Visualisations obligatoires

Les tableaux de chiffres ne suffisent pas à comprendre le comportement dynamique des modèles. Nous avons intégré les visualisations suivantes :

- Matrice de confusion : Indispensable pour identifier les classes souvent confondues (ex: erreur entre deux types de vêtements dans *Fashion-MNIST*).
- Courbes d'apprentissage (Loss et Accuracy) : Traçage de l'évolution de la perte et de l'exactitude sur les ensembles d'entraînement et de validation en fonction des époques.

6.3 Diagnostic par les courbes d'apprentissage

L'analyse des courbes d'apprentissage constitue notre outil de diagnostic principal:

- **Détection du surapprentissage (*Overfitting*)** : Si la perte d'entraînement continue de diminuer tandis que la perte de validation stagne ou augmente, nous avons diagnostiqué un surapprentissage. Cela indique que le modèle mémorise le bruit du dataset d'entraînement au lieu d'apprendre des motifs généralisables.
- **Détection du sous-apprentissage (*Underfitting*)** : Si les deux courbes stagnent à un niveau de perte élevé dès le début, le modèle manque de complexité ou les hyperparamètres (learning rate) ne permettent pas une descente efficace.
- **Stabilité de la convergence** : Une courbe lisse montre une optimisation stable, tandis que des oscillations fortes suggèrent un taux d'apprentissage trop élevé ou un besoin de normalisation des données.

7. Interprétabilité des modèles

L'interprétabilité permet de passer de la "boîte noire" à une compréhension des mécanismes de décision. Pour nos modèles, nous avons exploré deux approches :

- Analyse des poids (MLP) : En visualisant les poids de la première couche du MLP, nous avons pu identifier les caractéristiques les plus déterminantes pour le diagnostic. Les neurones présentant les poids les plus élevés correspondent aux attributs cliniques les plus corrélés avec la malignité.
- Cartes d'activation (CNN) : Pour les CNN, nous avons utilisé des méthodes de visualisation pour

observer quelles zones de l'image activent le plus fortement les neurones de la couche de convolution finale. Cela confirme que le modèle se focalise bien sur la forme et les textures des vêtements et non sur le fond de l'image.

8. Bonnes pratiques et erreurs fréquentes

L'ingénierie du *Deep Learning* est une discipline empirique où la rigueur méthodologique est le facteur clé du succès.

8.1 Checklist avant entraînement

Pour éviter les échecs lors du lancement des entraînements, nous avons établi ce protocole de vérification :

1. Vérification des dimensions : S'assurer que les tenseurs d'entrée (input) correspondent aux attentes de la première couche du modèle.
2. Statut du modèle : Toujours définir model.train() ou model.eval() pour activer/désactiver correctement le *Dropout* et la *Batch Normalization*.
3. Réinitialisation des gradients : Utiliser systématiquement optimizer.zero_grad() avant chaque étape de rétropropagation pour éviter l'accumulation parasite des gradients.
4. Device agnostic : Vérifier que les tenseurs et le modèle sont déplacés sur le même périphérique (CPU ou GPU) via

8.2 Erreurs fréquentes

L'expérience montre que les erreurs les plus courantes sont souvent les plus subtiles :

- **Gradient Explosion** : Souvent causé par un *Learning Rate* trop élevé ou l'absence de *Gradient Clipping* sur les architectures récurrentes.
- **Data Leakage (Fuite de données)** : Utiliser des données de test lors de la phase de normalisation (ex: calculer la moyenne/écart-type sur le jeu complet au lieu du seul jeu d'entraînement), ce qui biaise indûment les performances.
- **Overfitting masqué** : Confondre une haute précision sur l'entraînement avec une intelligence réelle. C'est l'écart entre la perte d'entraînement et la perte de validation qui reste le juge de paix.

8.3. Structure type du code d'entraînement

▸ La qualité d'un modèle dépend de la rigueur de sa boucle d'entraînement. Nous avons adopté une architecture modulaire en PyTorch, garantissant la séparation claire entre la phase d'apprentissage (*Train*) et la phase d'évaluation (*Validation*).

Workflow de la boucle d'entraînement

Chaque époque de notre processus d'apprentissage suit la séquence algorithmique suivante :

1. **Préparation** : Activation du mode entraînement (model.train()) pour permettre au Dropout et à la Batch Normalization d'opérer.
2. **Passage en avant (*Forward pass*)** : Le batch de données est injecté dans le modèle pour générer les prédictions : output = model(input).
3. **Évaluation du coût** : Calcul de la perte via la fonction criterion(output, target).
4. **Réinitialisation des gradients** : Appel systématique à optimizer.zero_grad() pour effacer les gradients accumulés des itérations précédentes.
5. **Rétropropagation (*Backward pass*)** : Calcul des gradients par rapport à la perte :

loss.backward().

6. **Optimisation** : Mise à jour des poids du réseau selon l'algorithme choisi (ex: Adam) via optimizer.step().

Gestion de la validation

À la fin de chaque époque, nous basculons en mode évaluation (model.eval()) et utilisons le contexte torch.no_grad(). Cette pratique est cruciale car elle désactive le calcul des gradients, réduisant ainsi drastiquement la consommation mémoire et garantissant que les poids du modèle ne sont pas modifiés pendant l'évaluation.

Note technique : L'utilisation constante de model.train() et model.eval() est la garantie d'une évaluation cohérente. L'omission de ces commandes est une erreur fréquente menant à des mesures de performance biaisées lors de la phase de test.

9.Guide de rédaction du rapport

La qualité d'un rapport technique ne repose pas uniquement sur la complexité des modèles implémentés, mais sur la clarté de la communication des travaux réalisés et de la justification des décisions d'ingénierie.

9.1 Structure attendue

Pour assurer une lecture fluide et logique, le rapport doit respecter une progression structurée :

- Résumé (Abstract) : Synthèse concise présentant la problématique, la méthodologie adoptée et les résultats majeurs obtenus.
- Introduction : Présentation du contexte, des objectifs du projet et du positionnement par rapport à l'état de l'art.
- Méthodologie : Description détaillée des étapes (prétraitement, choix des architectures, processus d'entraînement et hyperparamètres).
- Résultats et Discussion : Analyse critique des performances via des métriques quantitatives et des interprétations qualitatives.
- Conclusion : Synthèse des acquis, limites rencontrées et perspectives d'évolution.

9.2 Critères de qualité rédactionnelle

Pour garantir une évaluation optimale, les critères suivants doivent être respectés :

- Rigueur terminologique : Utilisation précise du vocabulaire technique (ex: distinguer *fonction de perte* de *métrique de performance*).
- Sobriété et clarté : Chaque paragraphe doit exprimer une idée unique. Éviter le jargon inutile et les phrases trop complexes.
- Cohérence visuelle : Chaque figure, graphique ou tableau doit être numéroté, légendé et impérativement cité dans le texte (ex: *« Comme illustré en Figure X... »*).
- Esprit critique : Un ingénieur justifie ses choix. Il ne suffit pas de décrire ce qui a été fait, il faut expliquer *pourquoi* ce choix a été validé ou, le cas échéant, pourquoi il a été écarté après expérimentation.

9.3 Ce qu’il faut éviter

La crédibilité du rapport dépend également de l'élimination de certaines pratiques :

- Le "copy-paste" sans justification : Ne jamais intégrer de blocs de code ou de résultats sans expliquer leur fonction précise dans le pipeline de développement.

- L'absence d'analyse des échecs : Un modèle qui ne converge pas ou qui présente un *overfitting* marqué est une source d'analyse précieuse. Ne cachez pas vos résultats médiocres ; expliquez-en les causes probables.
- La confusion entre "Entraînement" et "Test" : Toute évaluation des performances doit exclusivement s'appuyer sur des données non vues par le modèle lors de l'apprentissage (jeu de test ou de validation).
- Le manque de définition : Tout acronyme ou concept avancé (ex: *Beam Search*, *Dropout*, *Gradient Clipping*) doit être défini lors de sa première occurrence.

10.Références bibliographiques recommandées

La maîtrise du *Deep Learning* repose sur l'étude des travaux ayant fondé la discipline. Les références suivantes sont essentielles pour approfondir les concepts abordés dans ce rapport.

10.1 Articles fondateurs

Ces articles constituent la base théorique des architectures traitées :

- **Sur les MLP et la rétropropagation :** Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors*. Nature. (L'article qui a popularisé l'algorithme de base).
- **Sur les CNN** : LeCun, Y., et al. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE. (La référence historique pour les architectures convolutives type LeNet).
- **Sur les LSTM** : Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation. (L'article fondateur résolvant le problème de la disparition du gradient).
- **Sur l'optimiseur Adam** : Kingma, D. P., & Ba, J. (2014). *Adam: A method for stochastic optimization*. ICLR. (La référence actuelle pour le choix des optimiseurs).

10.2 Ressources pédagogiques

Pour aller plus loin, nous recommandons les supports suivants, reconnus pour leur qualité didactique :

- Ouvrage de référence (La "Bible") : Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (Disponible gratuitement en ligne).
- Documentation officielle PyTorch : pytorch.org/docs. Indispensable pour la maîtrise des API et des bonnes pratiques d'implémentation.
- Plateformes de cours :
    - *DeepLearning.AI* (Andrew Ng) sur Coursera : Idéal pour comprendre les intuitions mathématiques derrière les réseaux de neurones.
    - *Fast.ai* : Approche "top-down" très efficace pour une mise en pratique immédiate par le code.